Guía de Integración — API GraphQL ARKEO Fleet

Catálogo de productos del ERP Seven

Guía para desarrolladores que van a **consumir** el API. Explica cómo conectarse, qué consultar, cómo paginar y filtrar, y cómo manejar errores. No requiere conocer la infraestructura detrás.

---

1. Lo mínimo para empezar

Dato	Valor
URL	`https://xetmcig7xzdvzey5boopm5ivaq.appsync-api.us-east-2.amazonaws.com/graphql`
Método	`POST`
Header 1	`Content-Type: application/json`
Header 2	`x-api-key: <TU_API_KEY>`

> Es una API **GraphQL**. Siempre se llama con `POST` y el cuerpo es un JSON con el campo `query` (y opcionalmente `variables`).

Prueba rápida (una línea con cURL):

```
curl -s -X POST "https://xetmcig7xzdvzey5boopm5ivaq.appsync-api.us-east-2
  -H "Content-Type: application/json" \
  -H "x-api-key: <TU_API_KEY>" \
  -d '{"query":"query { productos(page:1, pageSize:5) { data { codigo_pro
```

2. Operaciones disponibles

2.1. productos (catálogo general)

```
productos(
  page: Int = 1
  pageSize: Int = 100
  filter: ProductoFilter
  orderBy: ProductoOrderBy = CODIGO_PRODUCTO_ASC
): ProductoConnection!
```

2.2. productosInactivos (solo inactivos)

```
productosInactivos(
  page: Int = 1
  pageSize: Int = 100
  filter: ProductoInactivoFilter
  orderBy: ProductoOrderBy = CODIGO_PRODUCTO_ASC
): ProductoConnection!
```

2.3. sistemasSubsistemas (grupos únicos)

```
sistemasSubsistemas(
  orderBy: SistemaOrderBy = CODIGO_SISTEMA_ASC
): [SistemaSubsistema!]!
```

> Cada operación devuelve un objeto con dos partes:
>
> - `data`: la lista de productos/sistemas de la página pedida.

- `pagination`: los metadatos para saber cuántas páginas hay (solo en productos y productosInactivos).

En GraphQL **tú eliges qué campos recibir**. Pide solo lo que necesitas.

3. Campos de un producto

Campo	Tipo	Descripción
`codigo_producto`	String	Código único del producto
`nombre_producto`	String	Nombre/descripción
`codigo_tipo`	String	Código del tipo
`nombre_tipo`	String	Nombre del tipo
`estado`	String	`ACTIVO` o `INACTIVO`
`codigo_sistema`	String	Código de sistema (grupo)
`nombre_sistema`	String	Nombre de sistema
`codigo_subsistema`	String	Código de subsistema (subgrupo)
`nombre_subsistema`	String	Nombre de subsistema
`fecha_creacion`	String	Fecha ISO 8601
`fecha_actualizacion`	String	Fecha ISO 8601
`codigo_unidad`	String	Código de unidad de medida
`nombre_unidad`	String	Nombre de unidad (ej. UNIDAD)

`trabajo_repuesto`	String	`TRABAJO` o `REPUESTO`

4. Paginación

- `page` empieza en **1**.
- `pageSize` por defecto **100**, máximo **1000**.
- Para recorrer todo: empieza en `page: 1` y sigue incrementando mientras `pagination.hasNextPage` sea `true`.

Ejemplo:

```
query {
  productos(page: 1, pageSize: 50) {
    data { codigo_producto }
    pagination { page totalPaginas hasNextPage }
  }
}
```

> **Nota:** La query `sistemasSubsistemas` NO tiene paginación. Retorna todas las combinaciones únicas (43 items).

5. Filtros

Todos opcionales. Si envías varios, se combinan con **AND**.

Filtro	Tipo	Comportamiento
`codigo_producto`	String	Coincidencia **exacta**
`codigo_producto_contains`	String	Coincidencia **parcial**
`nombre_producto_contains`	String	Coincidencia **parcial**

`codigo_tipo`	String	Exacta
`estado`	ACTIVO / INACTIVO	Exacta (solo en productos)
`codigo_sistema`	String	Exacta
`codigo_subsistema`	String	Exacta
`codigo_unidad`	String	Exacta
`trabajo_repuesto`	TRABAJO / REPUESTO	Exacta

Ejemplo:

```
query {
  productos(
    page: 1
    pageSize: 20
    filter: {
      estado: ACTIVO
      trabajo_repuesto: REPUESTO
      nombre_producto_contains: "ACEITE"
    }
  ) {
    data { codigo_producto nombre_producto }
    pagination { totalRegistros }
  }
}
```

6. Usar variables (recomendado)

En lugar de concatenar valores en el texto de la query, usa variables. Es más seguro y legible:

Query:

```
query ListarProductos($page: Int!, $size: Int!, $filter: ProductoFilter)
  productos(page: $page, pageSize: $size, filter: $filter) {
    data {
      codigo_producto
      nombre_producto
      estado
    }
    pagination {
      page
      totalRegistros
      totalPaginas
      hasNextPage
    }
  }
}
```

Variables (JSON):

```
{
  "page": 1,
  "size": 50,
  "filter": { "estado": "ACTIVO" }
}
```

7. Formato de respuesta


```
{
  "data": {
    "productos": {
      "data": [
        {
          "codigo_producto": "70M0101001",
          "nombre_producto": "AA005- D/M TURBO MOTOR CAMBIAR EMPAQUES",
          "estado": "ACTIVO"
        }
      ],
      "pagination": {
        "page": 1,
        "pageSize": 100,
        "totalRegistros": 10126,
        "totalPaginas": 102,
        "hasNextPage": true
      }
    }
  }
}
```


8. Manejo de errores

> **Importante:** GraphQL **siempre responde HTTP 200** cuando la petición llega bien, incluso si hay un error de negocio. El error viene en un arreglo `errors`.

```
{
  "data": null,
  "errors": [
    {
      "message": "Error interno al consultar el catálogo de productos.",
      "errorType": "Lambda:Unhandled"
    }
  ]
}
```

Regla práctica para el cliente:

1. Si la respuesta trae `errors` → tratarlo como fallo.
2. Usar `data.productos` solo si `errors` no existe.

HTTP	Causa	Acción
200	OK (revisar `errors` igual)	Normal
401	Falta o es inválido el `x-api-key`	Revisar el header
400	Query mal formada	Revisar sintaxis GraphQL

9. Ejemplos de código

JavaScript / Node.js (fetch)

```
const URL = "https://xetmcig7xzdvzey5boopm5ivaq.appsync-api.us-east-2.ama
const API_KEY = "<TU_API_KEY>";

const query = `
  query ($page: Int!, $size: Int!, $filter: ProductoFilter) {
    productos(page: $page, pageSize: $size, filter: $filter) {
      data { codigo_producto nombre_producto estado }
      pagination { totalRegistros totalPaginas hasNextPage }
    }
  }`;

async function listarProductos() {
  const resp = await fetch(URL, {
    method: "POST",
    headers: { "Content-Type": "application/json", "x-api-key": API_KEY }
    body: JSON.stringify({
      query,
      variables: { page: 1, size: 50, filter: { estado: "ACTIVO" } },
    }),
  });
  const json = await resp.json();
  if (json.errors) throw new Error(json.errors[0].message);
  return json.data.productos;
}

listarProductos().then((r) => console.log(r.pagination, r.data.length));
```

Python (requests)

```
import requests

URL = "https://xetmcig7xzdvzey5boopm5ivaq.appsync-api.us-east-2.amazonaws
API_KEY = "<TU_API_KEY>"

query = """
query ($page: Int!, $size: Int!, $filter: ProductoFilter) {
  productos(page: $page, pageSize: $size, filter: $filter) {
    data { codigo_producto nombre_producto estado }
    pagination { totalRegistros totalPaginas hasNextPage }
  }
}
"""

resp = requests.post(
    URL,
    headers={"Content-Type": "application/json", "x-api-key": API_KEY},
    json={"query": query, "variables": {"page": 1, "size": 50, "filter":
    timeout=40,
)
payload = resp.json()
if "errors" in payload:
    raise RuntimeError(payload["errors"][0]["message"])

productos = payload["data"]["productos"]
print(productos["pagination"]["totalRegistros"], "registros")
for p in productos["data"]:
    print(p["codigo_producto"], p["nombre_producto"])
```

10. Nuevas funcionalidades

10.1. Consultar productos inactivos

```
{
  "query": "query { productosInactivos(page: 1, pageSize: 10) { data { co
}
```

10.2. Obtener sistemas y subsistemas únicos

```
{
  "query": "query { sistemasSubsistemas { codigo_sistema nombre_sistema c
}
```

Beneficios de sistemasSubsistemas:

- ✅ Sin duplicados (SELECT DISTINCT)
- ✅ Sin paginación (retorna todo)
- ✅ Perfecto para poblar filtros en UI
- ✅ 43 combinaciones únicas disponibles

11. Preguntas frecuentes

¿Por qué GraphQL siempre devuelve 200 aunque falle?
Es el comportamiento estándar de GraphQL. Revisa siempre el campo `errors`.

¿Puedo pedir solo un campo?
Sí. Pide exactamente los campos que uses; la respuesta no incluirá el resto.

¿Hay límite de registros totales?
No. El límite es por página (`pageSize` máx 1000). Recorre todo paginando.

¿Cómo sé cuántas páginas hay?
En `pagination.totalPaginas`, o avanza mientras `hasNextPage` sea `true`.

El `x-api-key` tiene vencimiento.
Sí, es una key de prueba. Si te da 401, solicita una key vigente al equipo que administra el API.

API GraphQL ARKEO Fleet - Catálogo de Productos
Versión 2.0 - Octubre 2026
Región: us-east-2 (Ohio)